UNIVERSITÄT PASSAU

Software Analysis

Introduction

Gordon Fraser
Lehrstuhl für Software Engineering II

Contents

- What is Software Analysis?
- Course Organisation
- First Steps: Character Level Analysis

What is Software Analysis?

The process of extracting information about a program

From its source code or artefacts
(e.g., Java bytecode, execution traces, Git history, …)

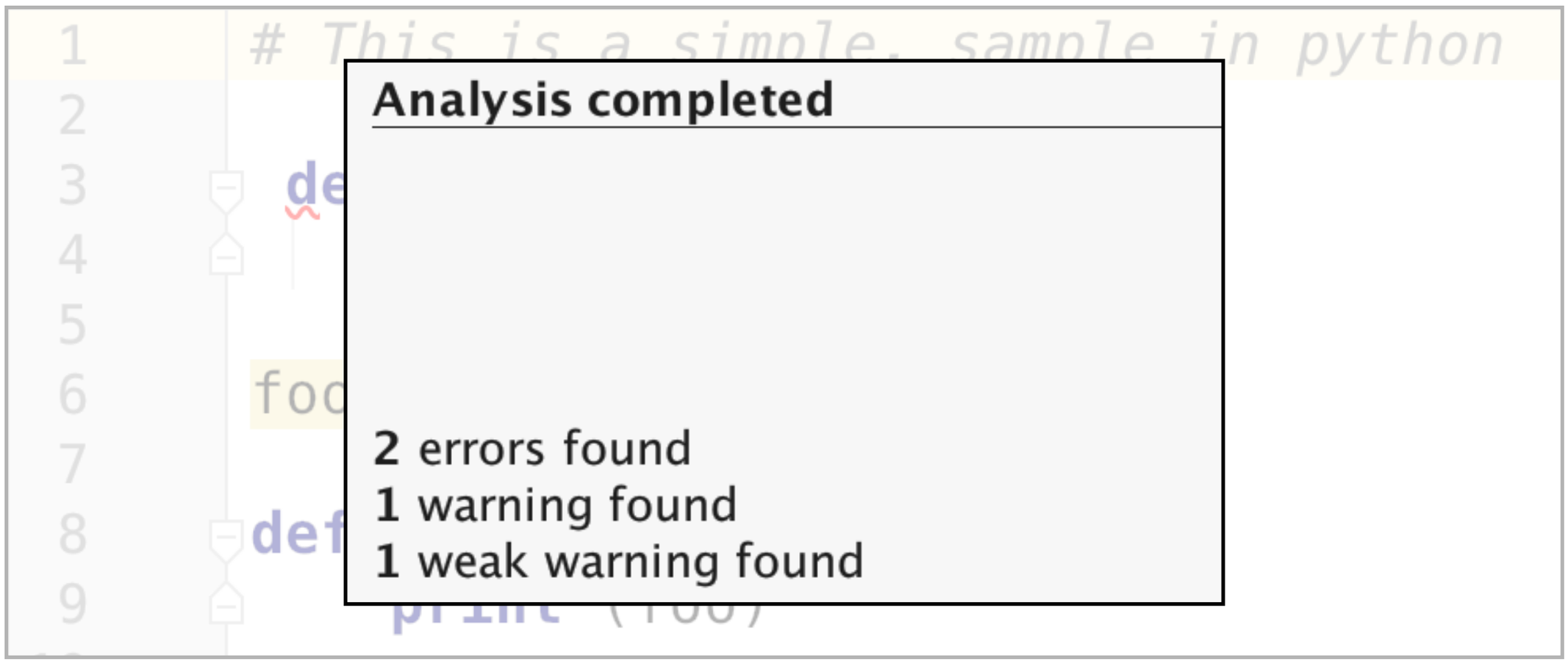
Information is extracted using automatic tools

Example

```
1  # This is a simple, sample in python
2
3  def foo():
4          pass
5
6  foo = None
7
8  def bar(foo):
9      print (foo)
```

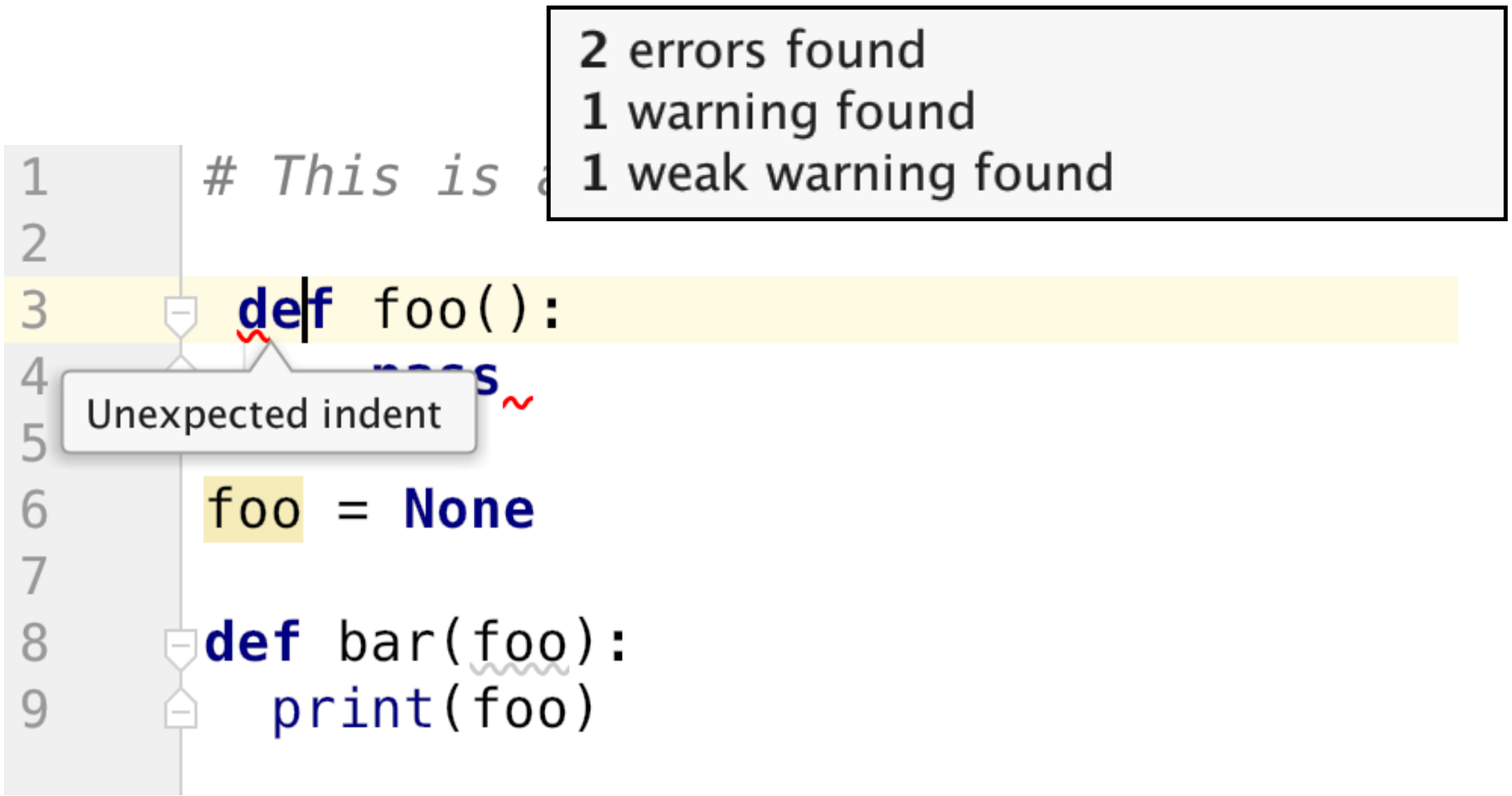
Can you tell me what's wrong?

Example



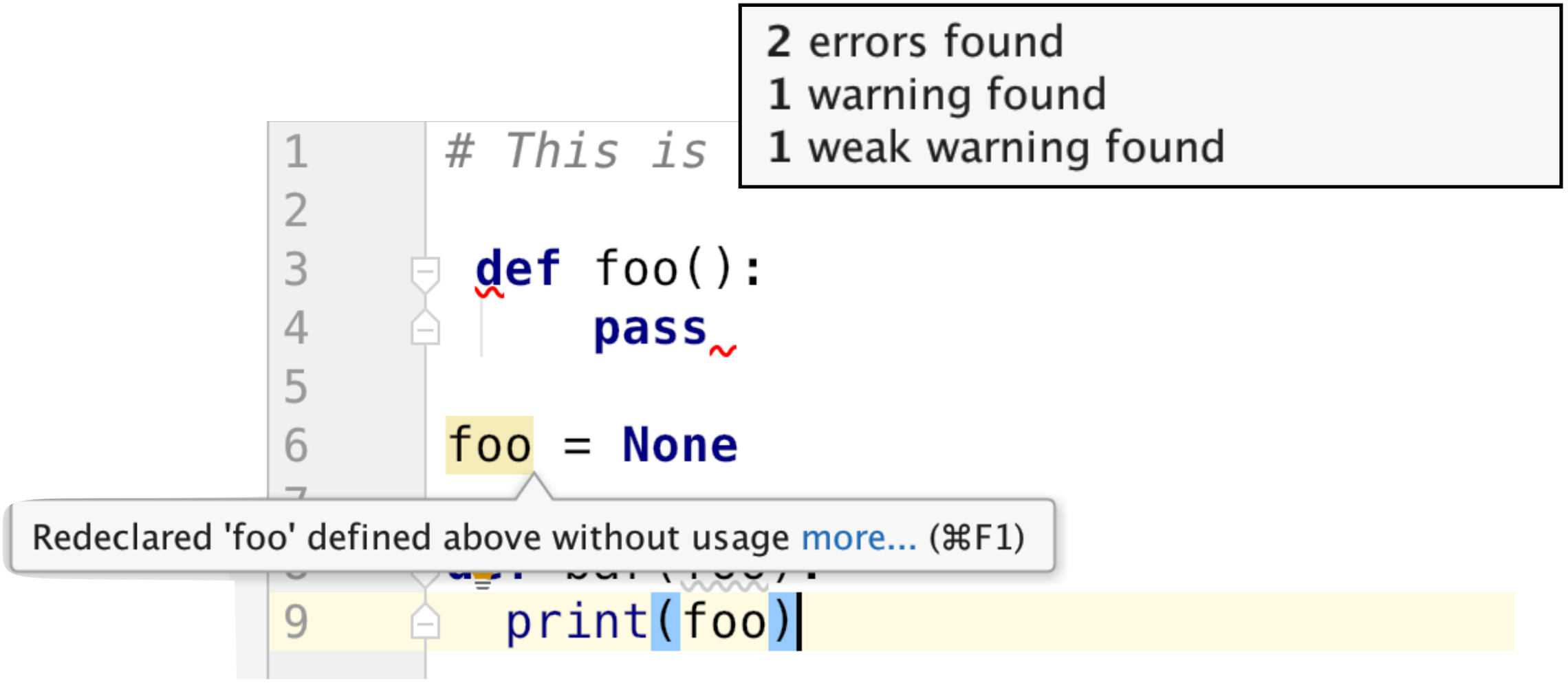
Can you tell me what's wrong?

Example



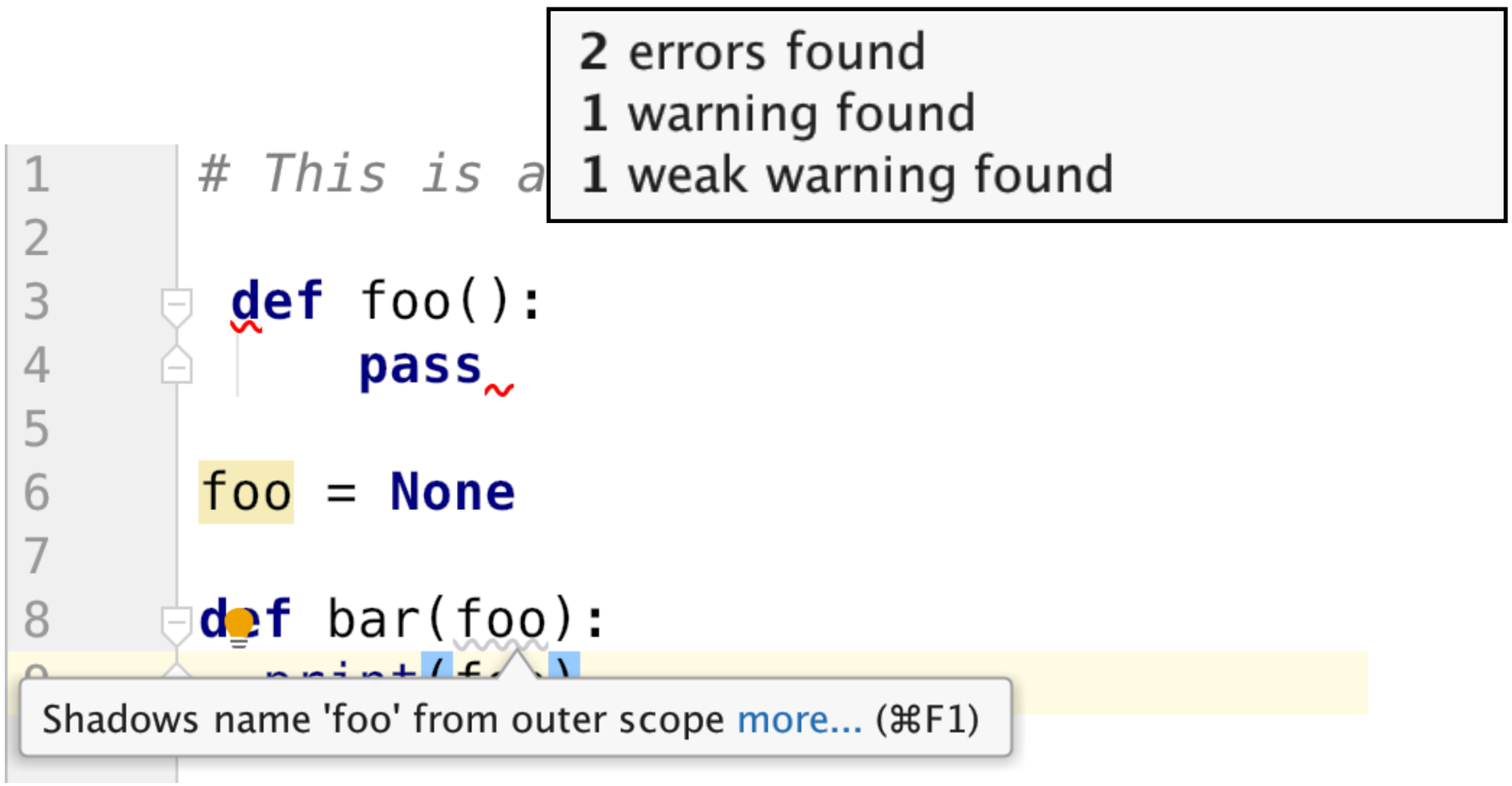
Can you tell me what's wrong?

Example



Can you tell me what's wrong?

Example



Can you tell me what's wrong?

Goals of Software Analysis

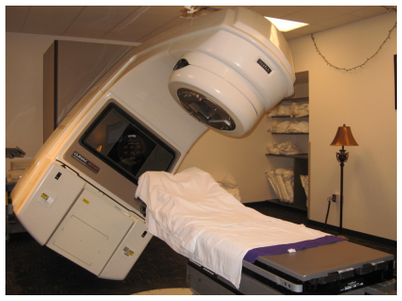
- **Establish** whether given properties hold for some software

- **Structural properties** must hold at design time:
  - *the code is indented using tabs*
  - *class* Bar *has only visible attributes*
  - *module* baz *contains more than 500 lines of code*

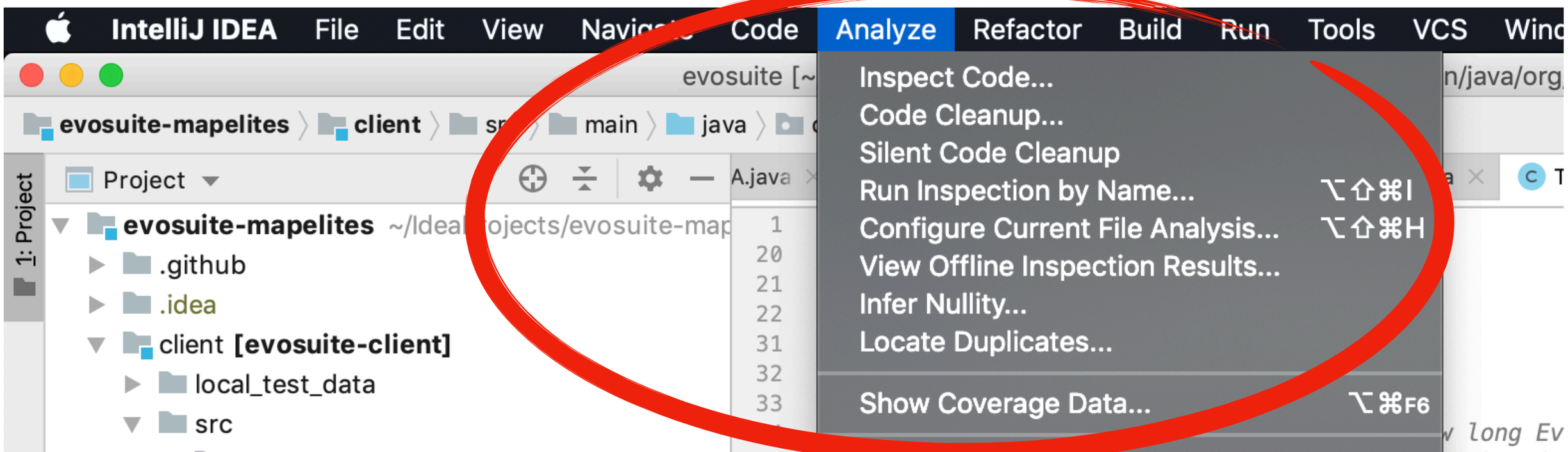
- **Behavioural properties** must hold while the software is running:
  - *method* foo *always terminates*
  - *in 80% of the runs, statement* xyz *executes in less than 10 ms*
  - *the program crashes with input* 42

Why Software Analysis?

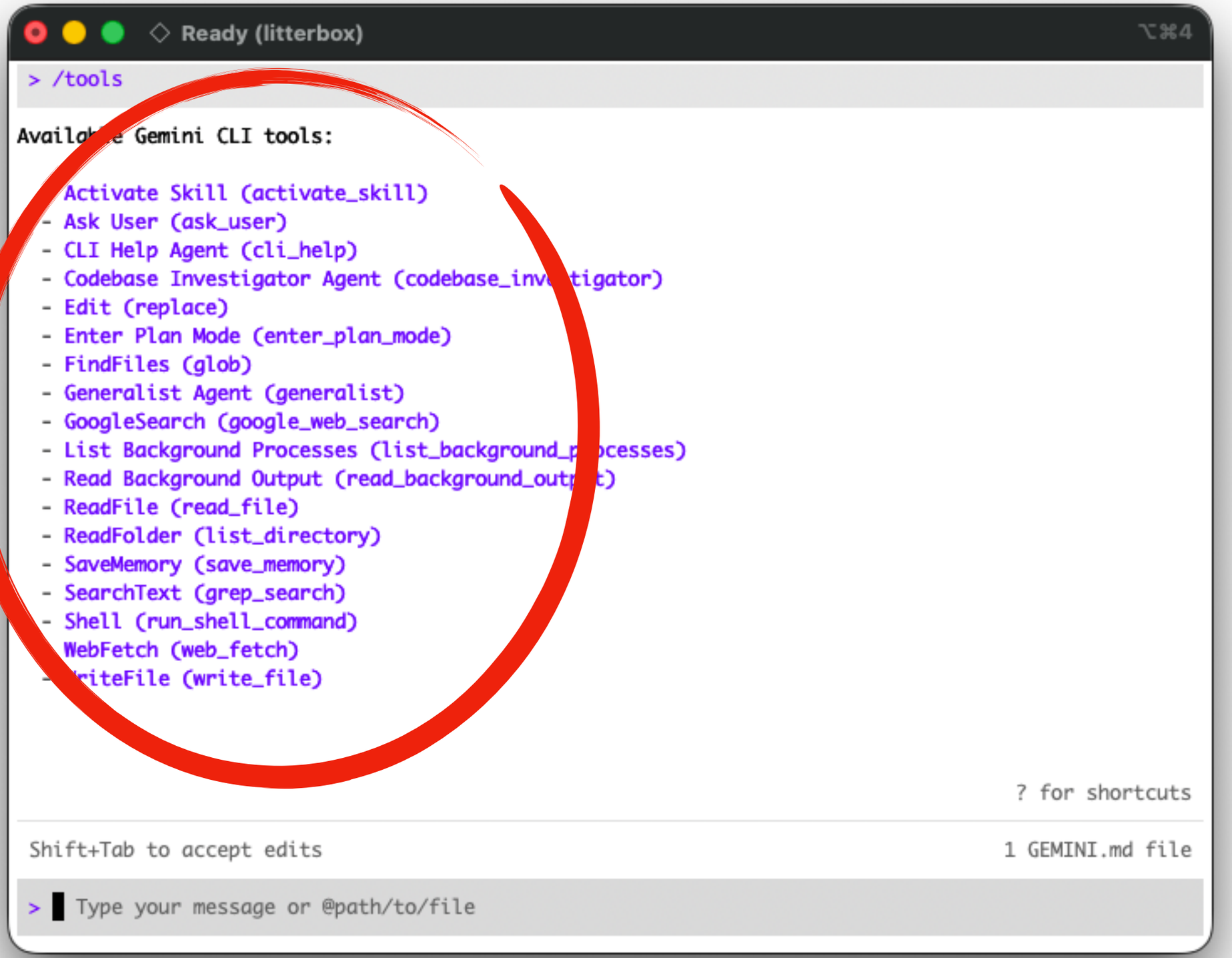
<Insert generic **software quality** motivation here>

<Insert generic **programmer productivity** motivation here>



Why Software Analysis?



Hierarchy of Analysis

- *Deduction*
- *Observation*
- *Induction*
- *Experimentation*

Deduction

- Deduction is reasoning from the general to the particular; it lies at the core of all reasoning techniques.

- In program analysis, deduction is used for reasoning **from the program code (or other abstractions) to concrete runs** for deducing what can or cannot happen.

- Deduction does not require any knowledge about the concrete runs; hence, the analysis is **static**

Deduction

```
1 char* format = “a = %d”;
2 if (p)
3   a = compute_value();
4 sprintf(buf, format, a);
```

Assume that buf contains the string “a = 0” (an error)

Observation

- Observation allows the programmer to inspect arbitrary aspects of an individual program run.

- Since an actual run is required, the associated techniques are called **dynamic**.

- Observation brings in actual facts of a program execution; unless the observation process is flawed, these facts cannot be denied.

Observation

```
1 char* format = “a = %d”;
2 if (p)
3   a = compute_value();
4 sprintf(buf, format, a);
```

Induction

- Induction is reasoning from the particular to the general.
- In program analysis, induction is used to **summarize multiple program runs to some abstraction that holds for all considered program runs (i.e., invariants)**
- A "program" may also be a piece of code, like a function or loop body.

Induction

```
1 char* format = “a = %d”;
2 if (p)
3   a = compute_value();
4 sprintf(buf, format, a);
```

Example **dynamic** invariant: a < 2054567 || a % 2 == 1

Experimentation

- Many problems in program understanding can be formulated as a search for causes:
  - *What is the cause for buf containing "a = 0”?*

- To prove actual causality, one needs **two experiments**: one where cause and effect occur, and one where neither cause nor effect occur.

- The cause must precede the effect, and the cause must be a minimal difference between these experiments.

- Searching for the actual cause thus requires a series of experiments, refining and rejecting hypotheses until the actual cause is isolated.

Experimentation

```
1 char* format = “a = %d”;
2 if (p)
3   a = compute_value();
4 sprintf(buf, format, a);
```

Experimentation

```
1 char* format = “a = %d”;
2 if (p)
3   a = compute_value();
4 sprintf(buf, format, a);
```

“a = %d” but a is a float

Experimentation

```
1 char* format = “a = %f”;
2 if (p)
3   a = compute_value();
4 sprintf(buf, format, a);
```

“a = %d” but a is a float

Hierarchy of Analysis

- ***Deduction*** from an abstraction into the concrete
  For instance, analysing program code to deduce what can or cannot happen in concrete runs.

- ***Observation*** of concrete events
  e.g. tracing, monitoring or profiling a program run or using a debugger.

- ***Induction*** for summarising observations into an abstraction
  an invariant, for example, or some visualisation.

- ***Experimentation*** for isolating causes of given effects
  e.g. narrowing down failure-inducing circumstances by systematic tests.

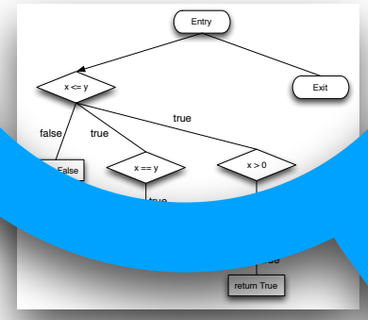
Software Analysis

Executable Program

Execution Trace

```
0101011010
1010110101
0101101010
1010101010
1010111111
```

```
public class IntentionDAO {

    /**
     * Stores a given {@link DefenderIntention} in the database.
     * <p>
     * For context information, the associated {@link Test} of the intention
     * is provided as well.
     *
     * @param test      the given test as a {@link Test}.
     * @param intention the given intention as an {@link DefenderIntention}.
     * @return the generated identifier of the intention as an {@code int}.
     * @throws Exception If storing the intention was not successful.
     */
    public static int storeIntentionForTest(Test test, DefenderIntention intentio
        int testId = test.getId();
        int gameId = test.getGameId();

        final String targetLines = intention.getLines().stream().map(String::valu

        final String query = String.join( delimiter: "\n",
                ...elements: "INSERT INTO intention (Test_ID, Game_ID, Target_Lines
                "VALUES (?,?,?);"
        );

        DatabaseValue[] values = new DatabaseValue[]{
                DatabaseValue.of(testId),
                DatabaseValue.of(gameId),
                DatabaseValue.of(targetLines),
        };

        final int result = DB.executeUpdateQueryGetKeys(query, values);
        if (result != -1) {
            return result;
        } else {
```



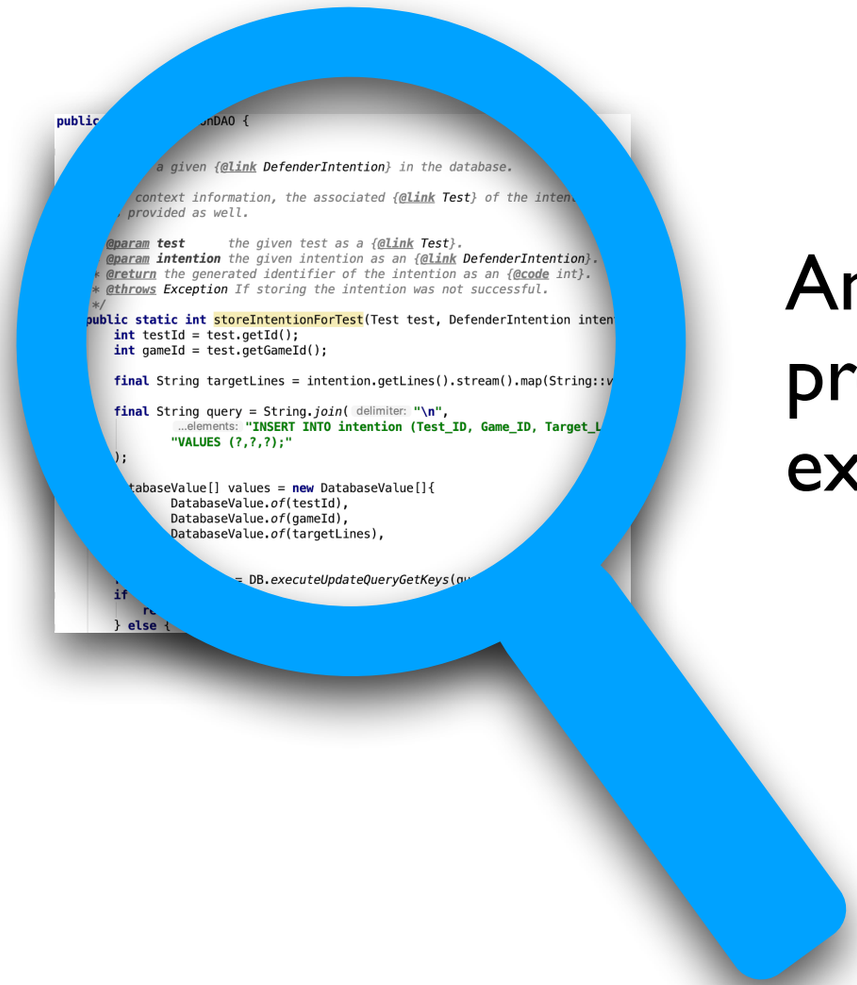
Source Code

Internal Representation

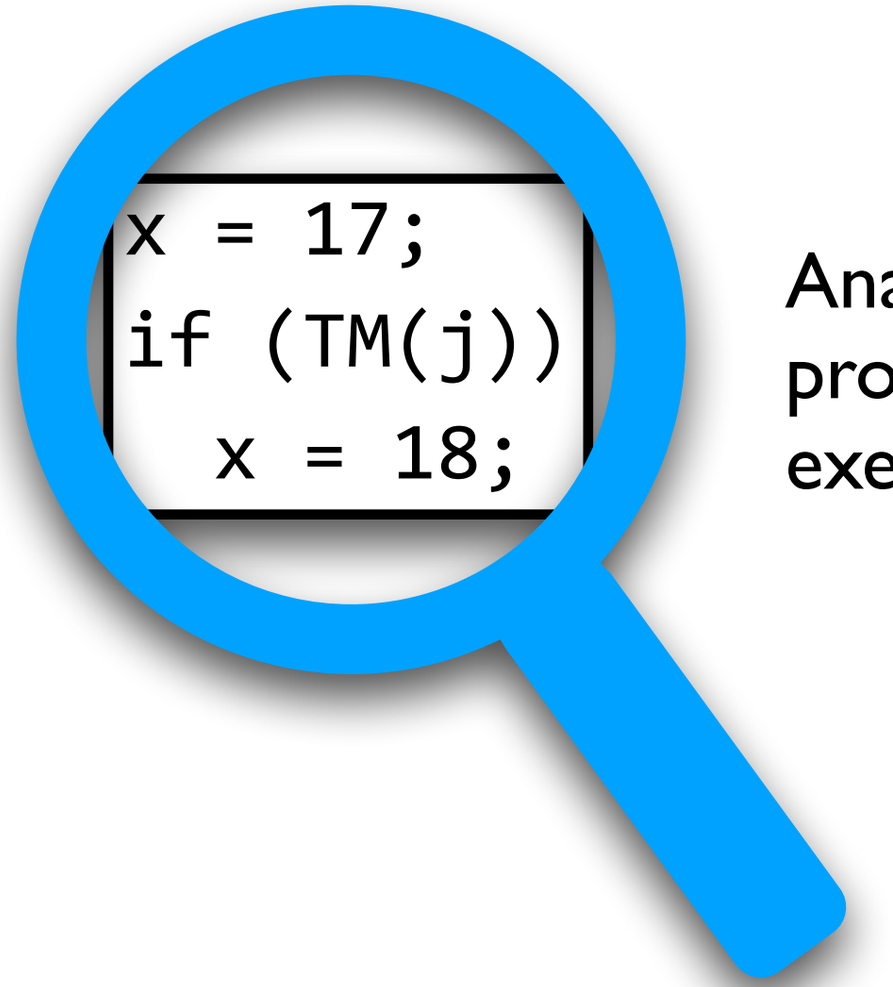
Analysis Algorithm

Software Analysis

- We've written a program P and we want to know...
- Does P satisfy property of interest ϕ?
  - *For example, P does not dereference a null pointer, all casts in P are safe, ...*
- Manually verifying that P satisfies ϕ may be tedious or impractical if P is large or complex
- Would be great to write a program that can inspect the source code of P and determine if ϕ holds!



Analyser that decides if a variable in a program has a constant value in any execution



Analyser that decides if a variable in a program has a constant value in any execution

x has a constant value if and only if the j'th Turing machine does not halt on empty input. If the hypothetical constant-value analyser exists, then we have a decision procedure for the halting problem, which is **known to be impossible**.

Rice's Theorem

"*All non-trivial semantic properties of programs are undecidable*"

- **Non-trivial**: there exists both a program that has that property and one that does not
- **Undecidable**: no automated method can determine whether the property holds for any program

We can **abstract** the behaviours of the program into a **decidable approximation** and attempt the analysis in the abstract version of the program

Example Analysis

- *Given a program P, determine the sign (positive, negative, or zero) of all of its variables (**sign analysis**).*

- Applications:
  - Check for division by 0
  - Optimize by storing positive variables as unsigned integers
  - Check for negative values, e.g., negative array indices.

Example Approximation

- Abstract concrete domain of integer values into **abstract domain of signs** {⊖, ⊙, ⊕} ∪ {⊤,⊥}

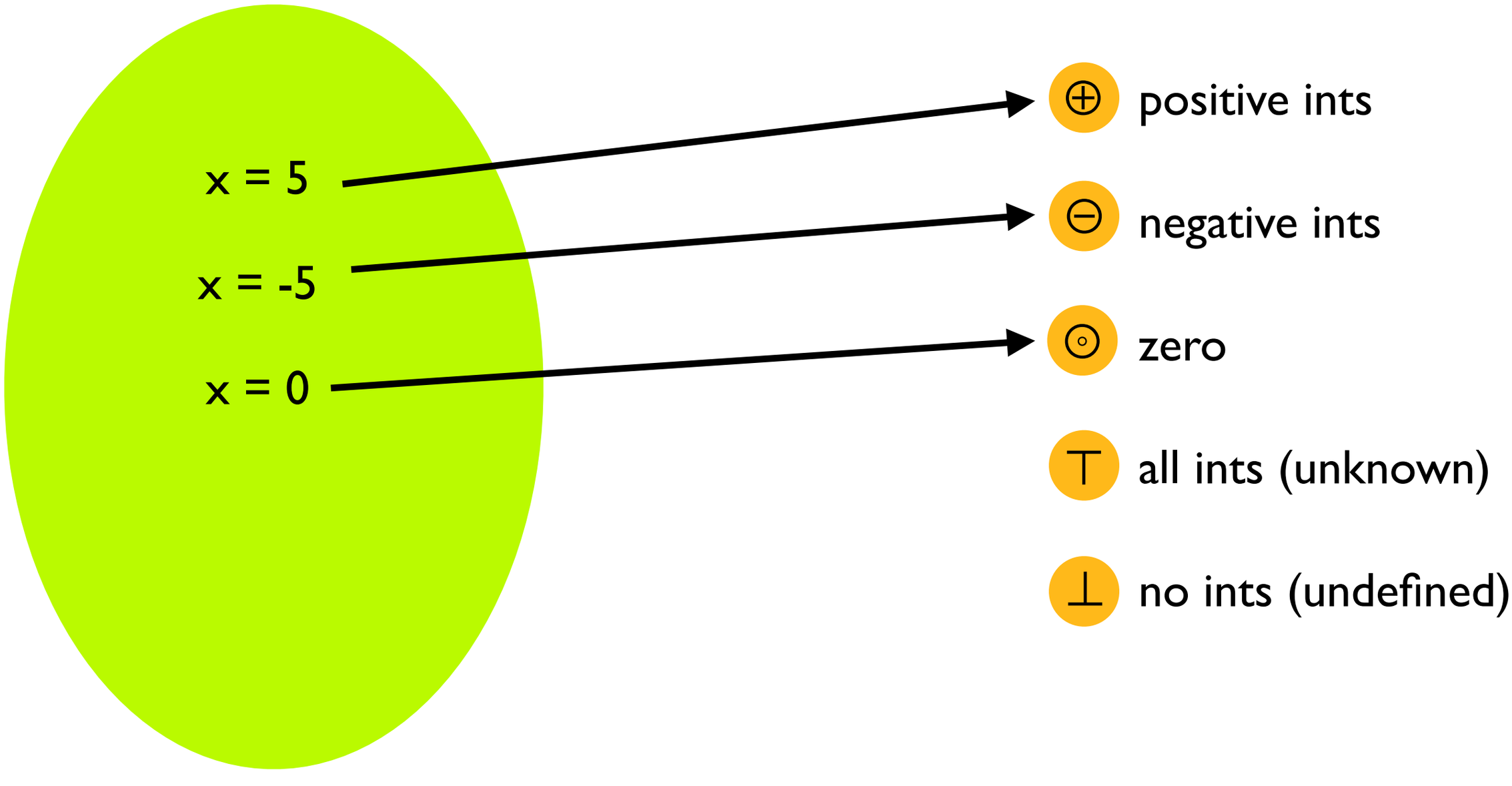
- Step 1: Define abstraction function abstract() over integer literals:

  - abstract(i) = ⊖ if i < 0
  - abstract(i) = ⊙ if i == 0
  - abstract(i) = ⊕ if i > 0

Example Analysis

Concrete domain of ints

Abstract domain of signs



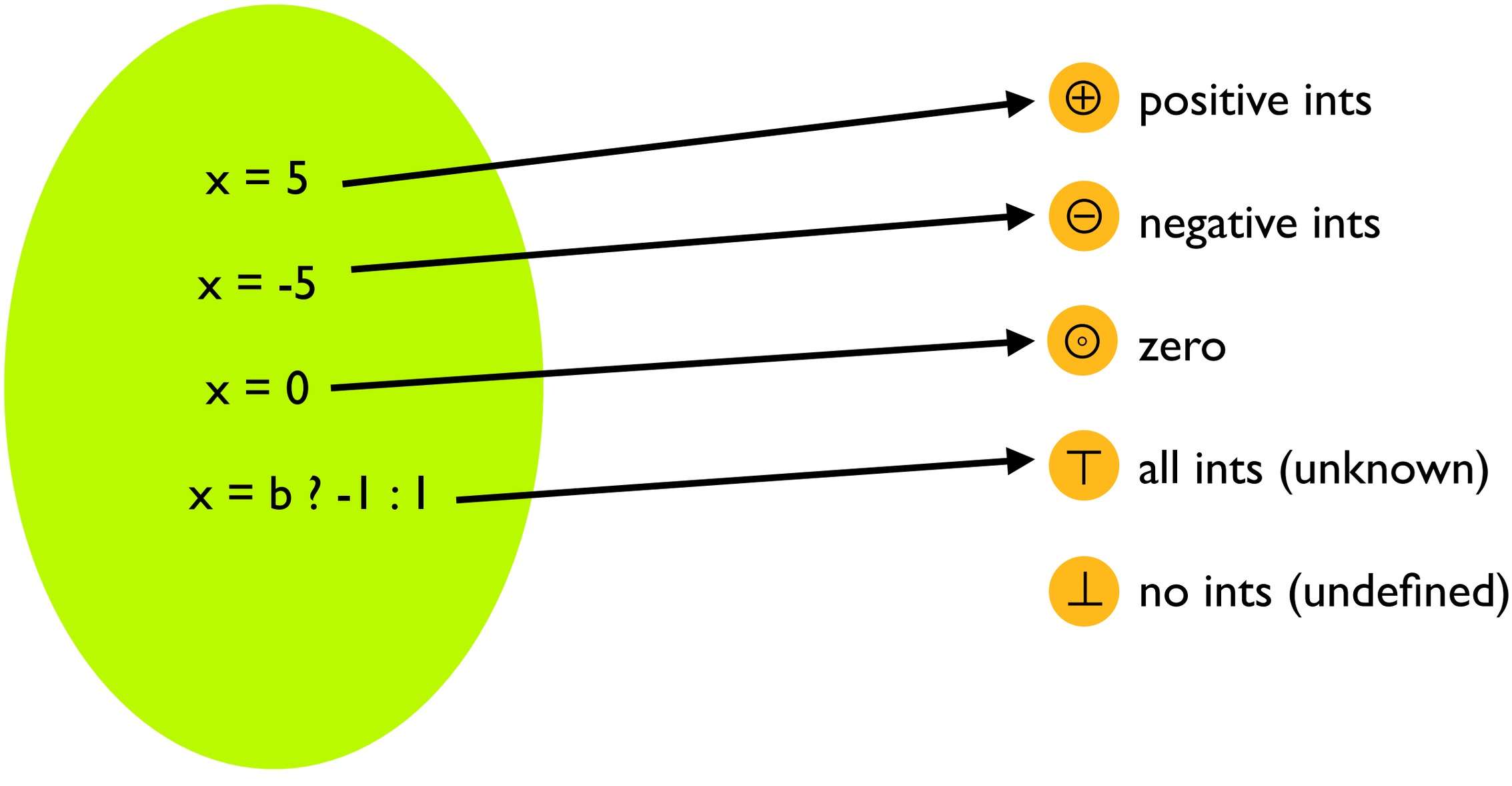
Example Analysis

- Define **transfer functions over the abstract domain** that show how to evaluate expressions in the abstract domain:
  - $\oplus + \oplus = \oplus$
  - $\ominus + \ominus = \ominus$
  - $\odot + \odot = \odot$
  - $\odot + \oplus = \oplus$
  - $\odot + \ominus = \ominus$
  - … …
  - $\oplus + \ominus = \top$
  - $\{\oplus, \ominus, \odot, \top\} + \top = \top$
  - $\{\oplus, \ominus, \odot, \top\} / \odot = \bot$

Example Analysis

Concrete domain of ints

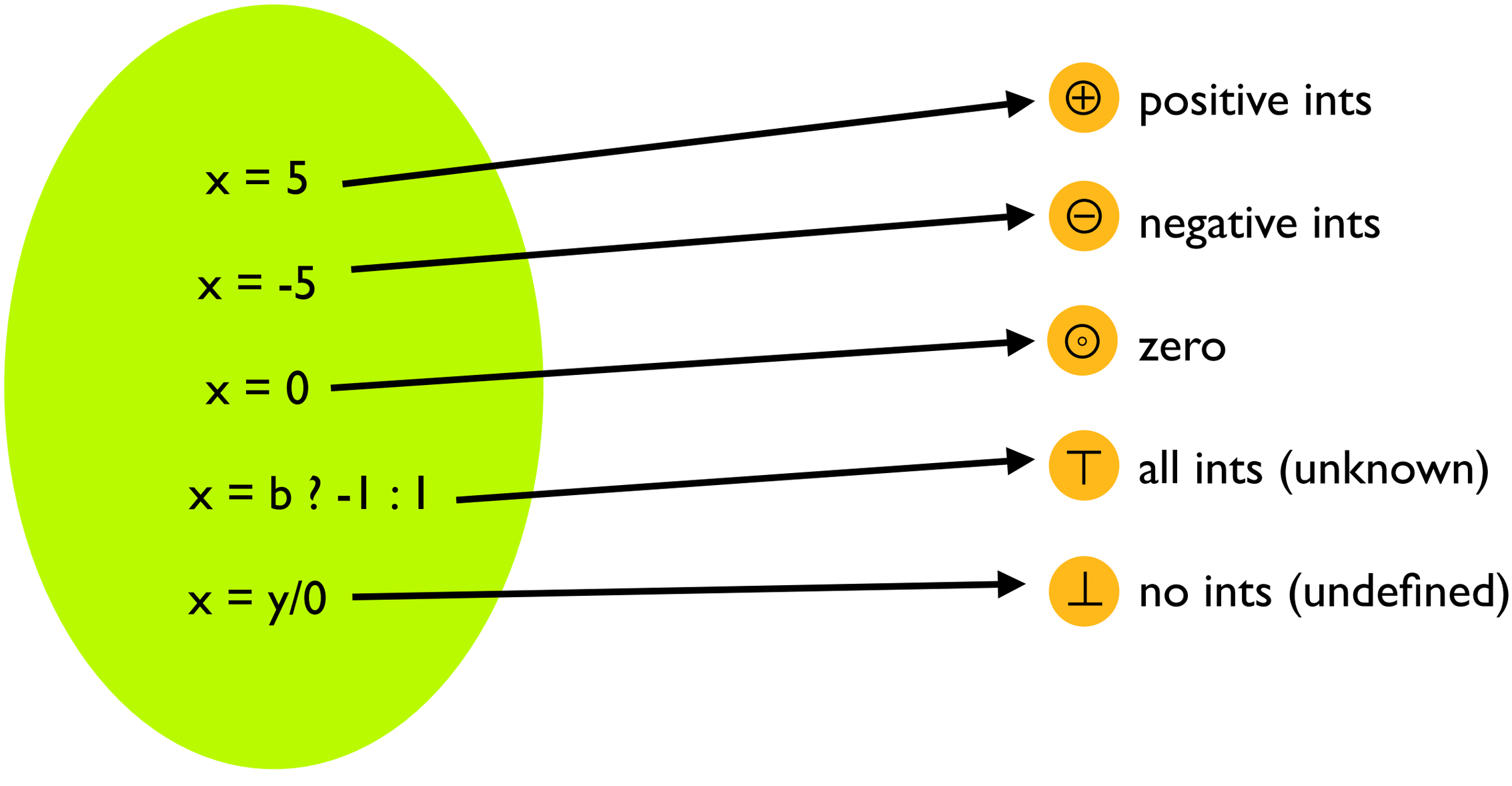
Abstract domain of signs



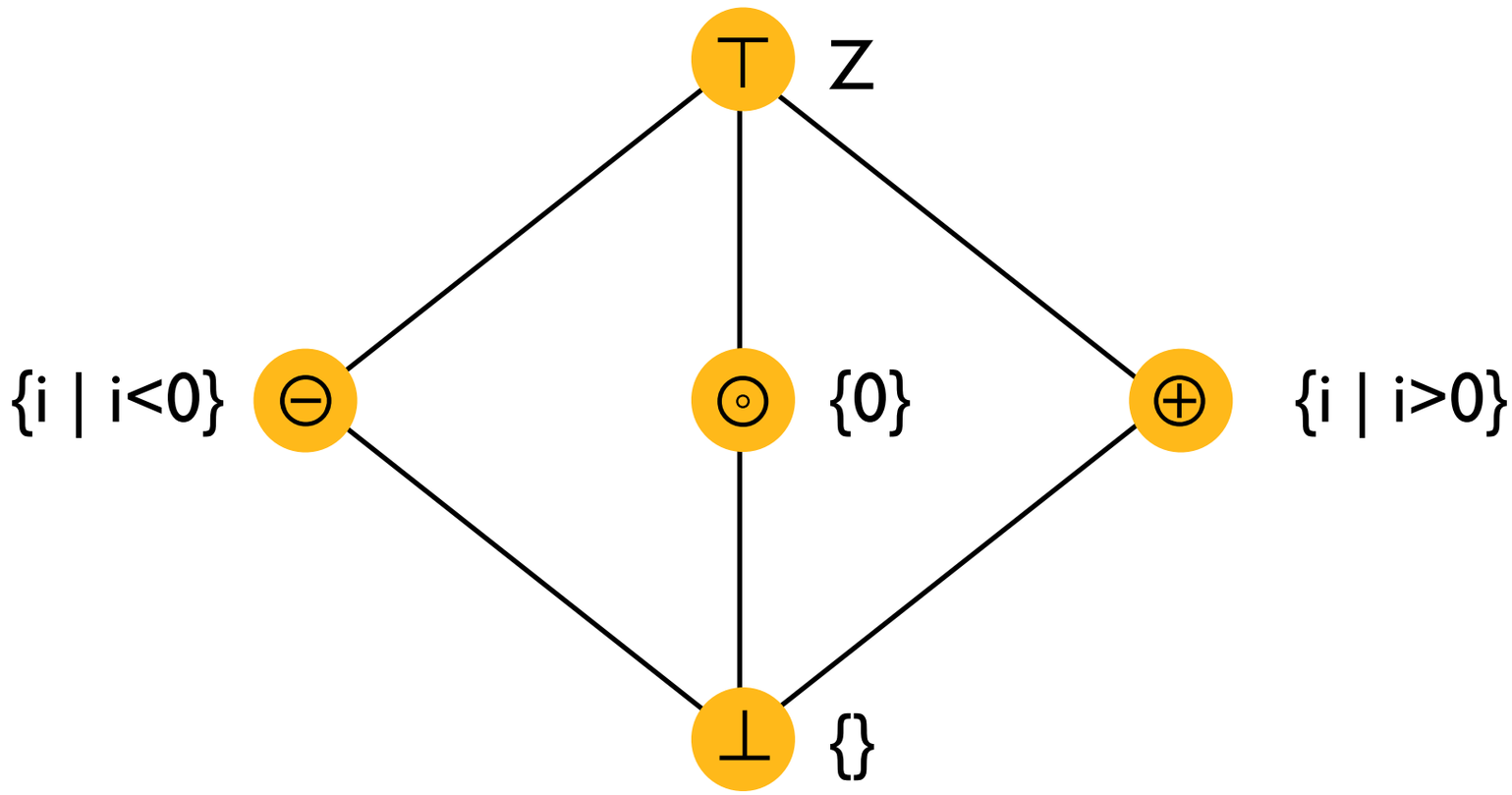
Example Analysis

Concrete domain of ints

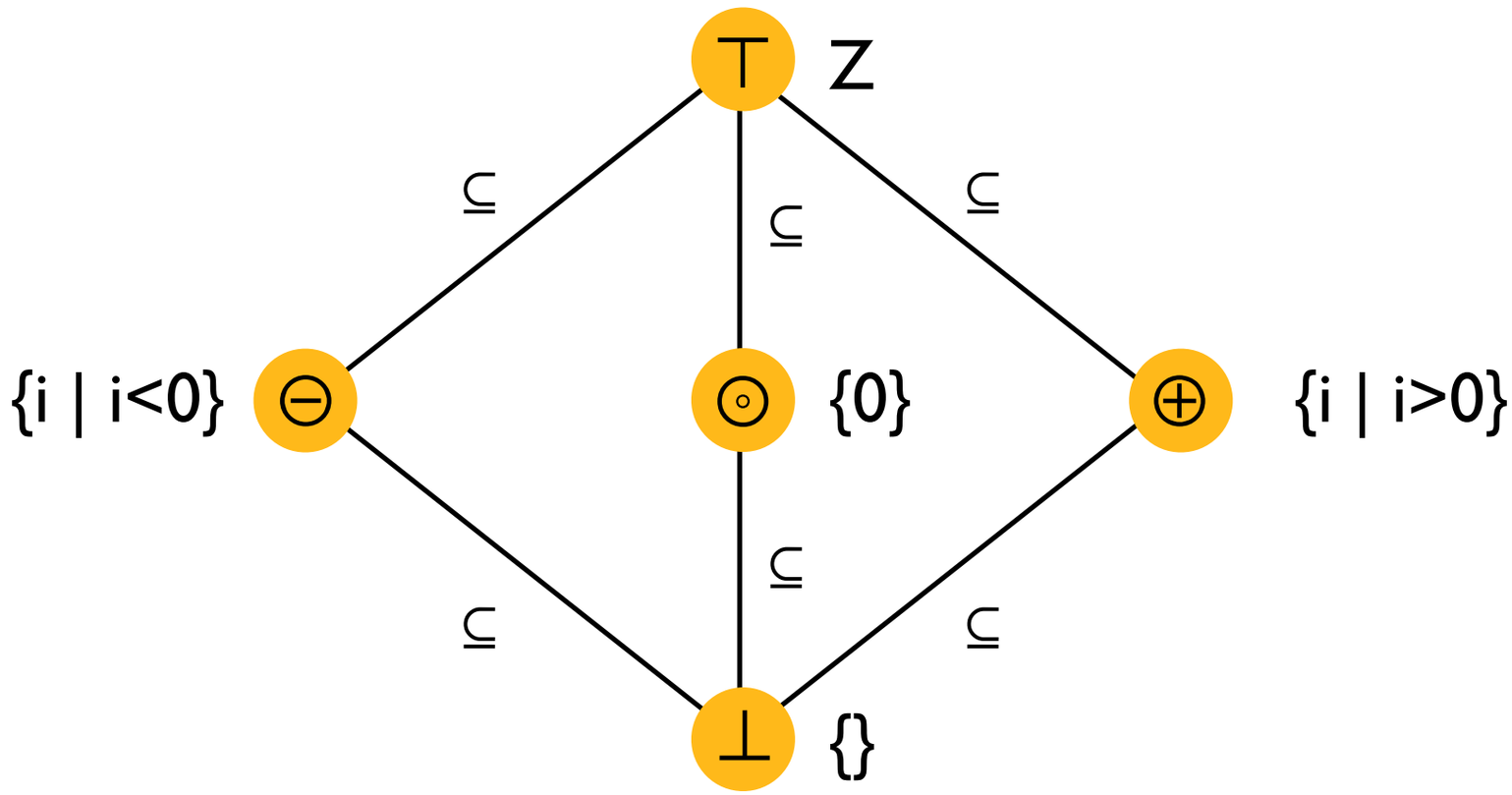
Abstract domain of signs



Example Analysis



Example Analysis



Example Approximation

Now, every integer value and expression has been abstracted into a {⊖, ⊙, ⊕, ⊤, ⊥}. This is very simplistic (so easy to solve) but even so it is useful. For example:

- Check for division by zero by looking for occurrences of ⊥

- Optimize the program by storing:
  - ⊕ variables as unsigned integers
  - ⊙ variables false boolean literals

Example Analysis

```
a = 5;
b = -3;
c = a × b;
d = 0;
e = c × d;
f = 10 / e;
```

```
a = ⊕;
b = ⊖;
c = ⊖;
d = ⊙;
e = ⊙;
f = ⊥;
```

Detected division by zero! Just look for variables that the analysis maps to ⊥.

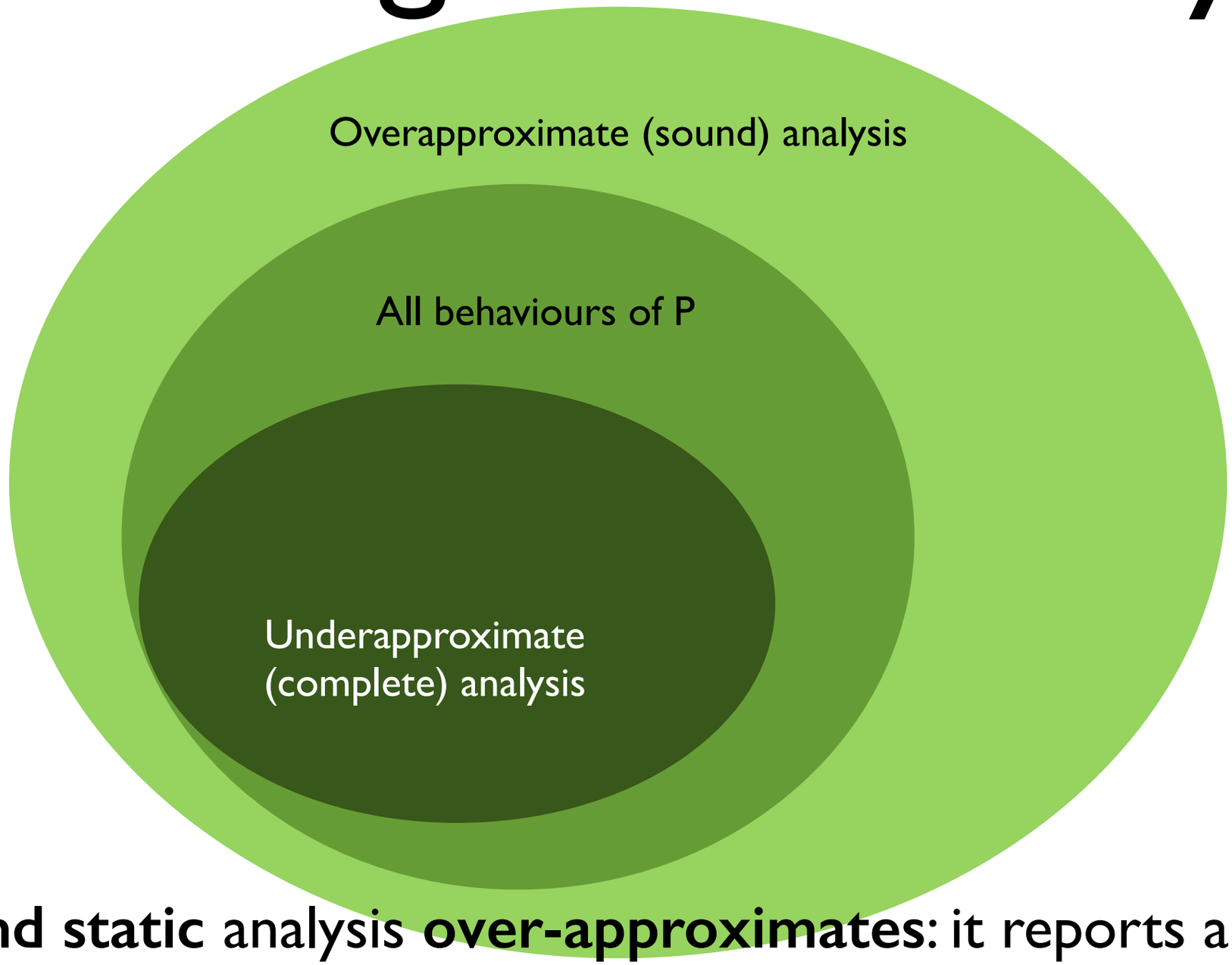
Example Analysis

```
a = 5;
b = -3;
c = a + b;
d = 0;
e = c - d;
f = 10 / e;
```

```
a = ⊕;
b = ⊖;
c = ⊤;
d = ⊙;
e = ⊤;
f = ⊤;
```

False positive! This program can never throw an error, but the analysis reports that f may contain any value (including undefined)

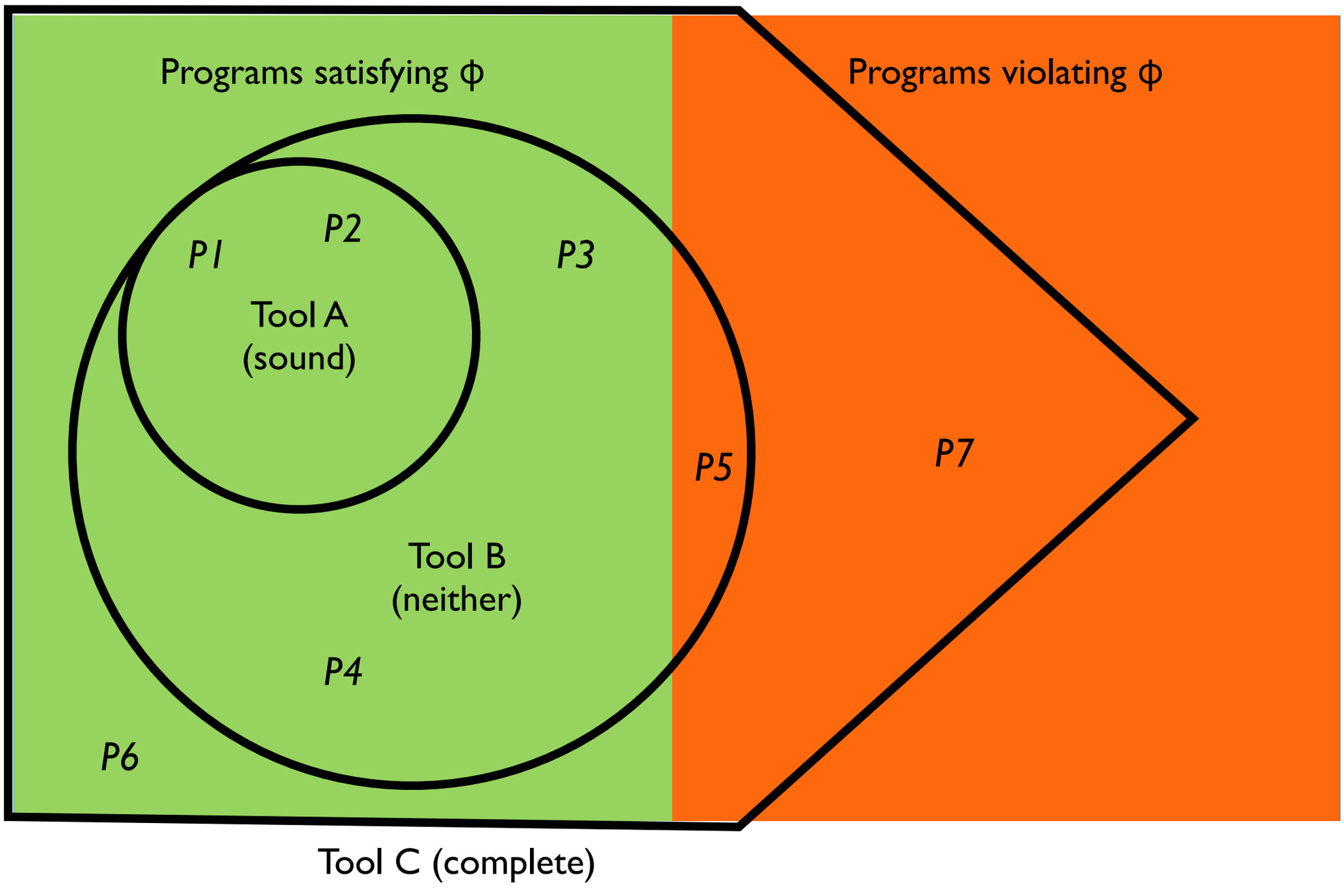
Does Program P Satisfy ϕ?



A **sound static** analysis **over-approximates**: it reports all the errors but may report some "false alarms".

A **complete static** analysis **under-approximates**: it reports only real errors but may not report all of them.

Which Programs Satisfy ϕ?



Soundness and Completeness

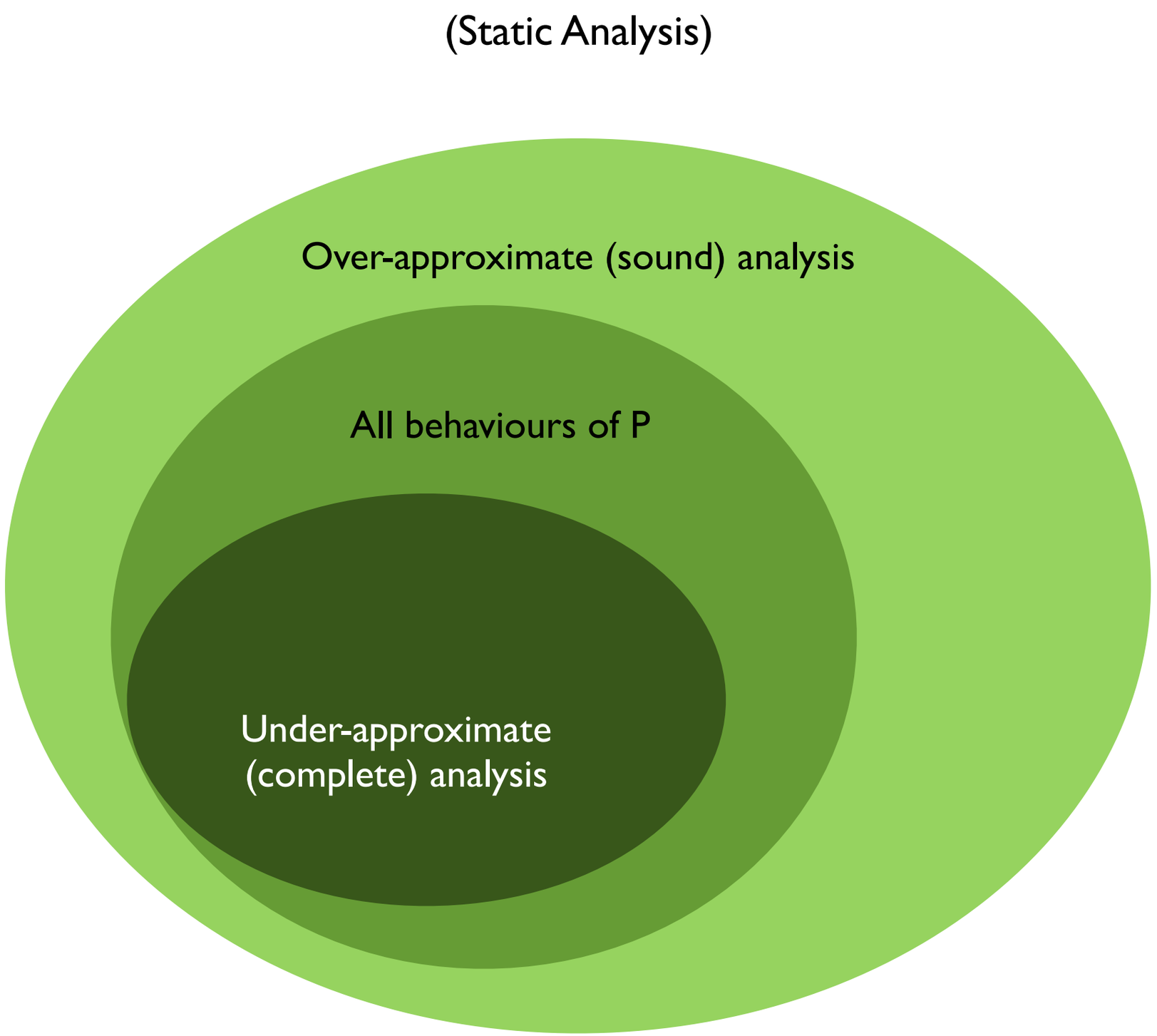
	Complete	Incomplete
Sound	Reports all errors Reports no false alarms Undecidable	Reports all errors May report false alarms Decidable
Unsound	May not report all errors Reports no false alarms Decidable	May not report all errors May report false alarms Decidable

Soundiness

- Static analysis tools that give up soundness **under specific circumstances** but still achieve few false alarms are defined *soundy*

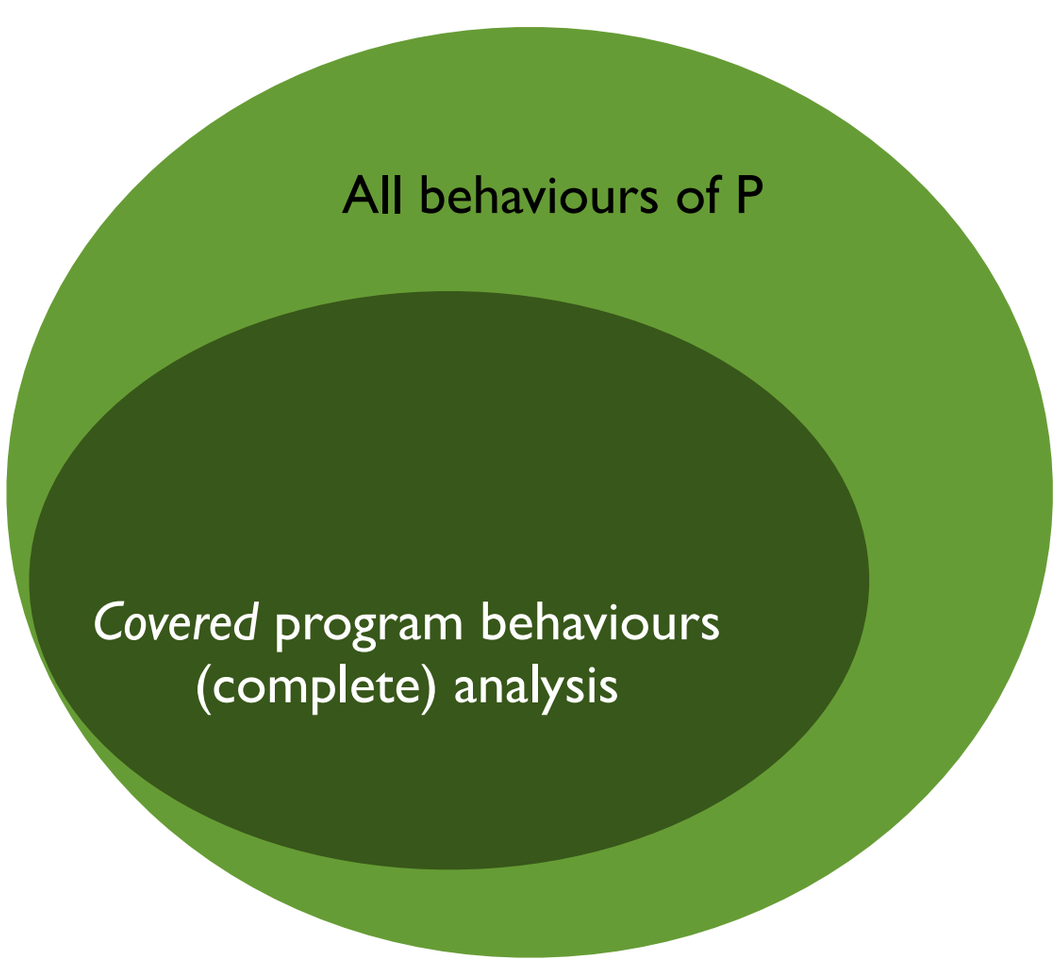
- *Soundy* tools cannot handle specific features (e.g., reflection) but can handle correctly programs that do not make use of those feature (aim for the common case)

Does Program P Satisfy ϕ?



Does Program P Satisfy ϕ?

(Dynamic Analysis)



Example Analysis

```
a = 5;
b = -3;
c = a + b;
d = 0;
e = c - d;
f = 10 / e;
```

```
a = ⊕;
b = ⊖;
c = ⊤;
d = ⊙;
e = ⊤;
f = ⊤;
```

False positive! This program can never throw an error, but the analysis reports that f may contain any value (including undefined)

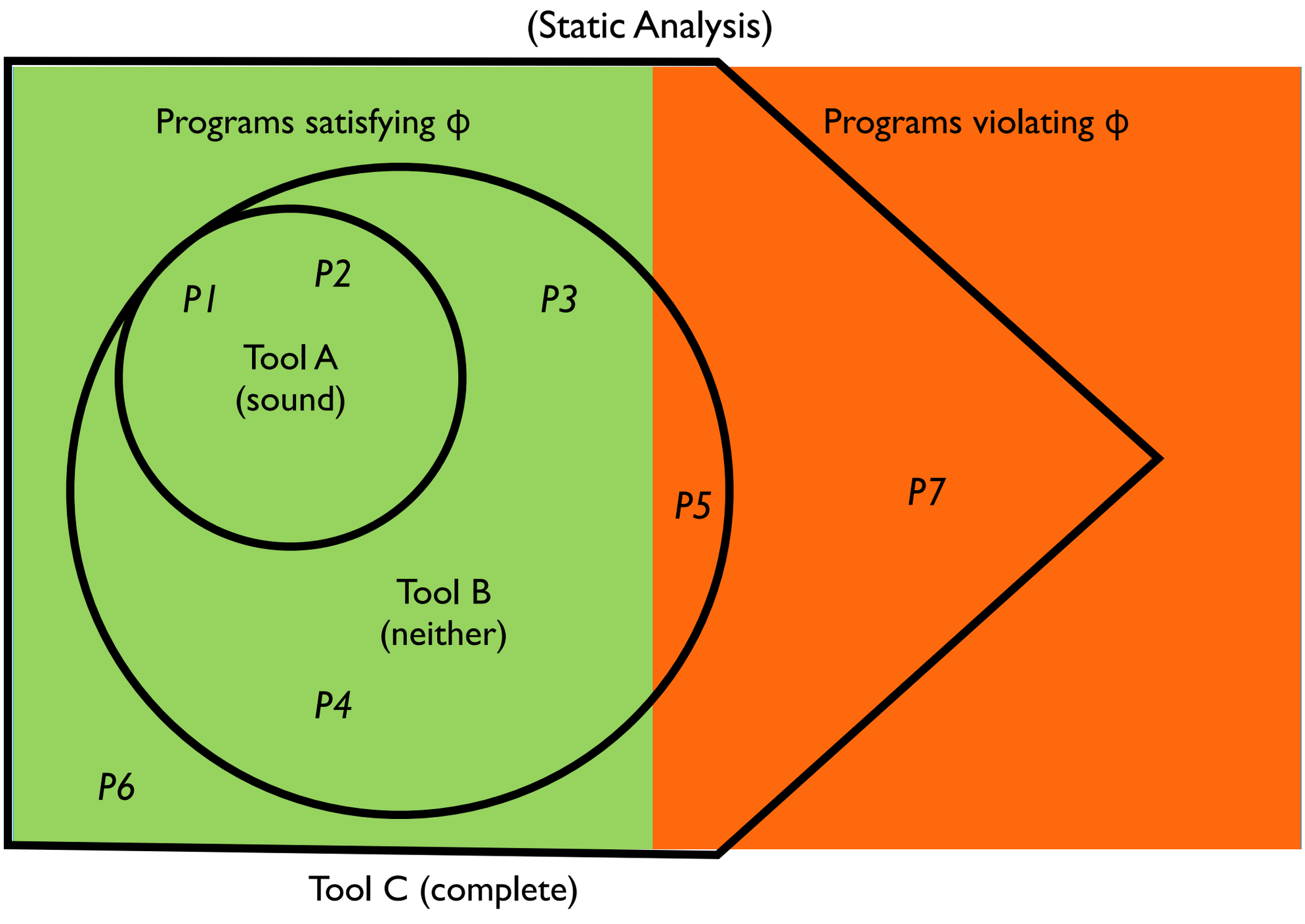
Dynamic Analysis

```
def foo(a):
    b = -3
    c = a + b
    d = 0
    e = c - d
    f = 10 / e
    return f
```

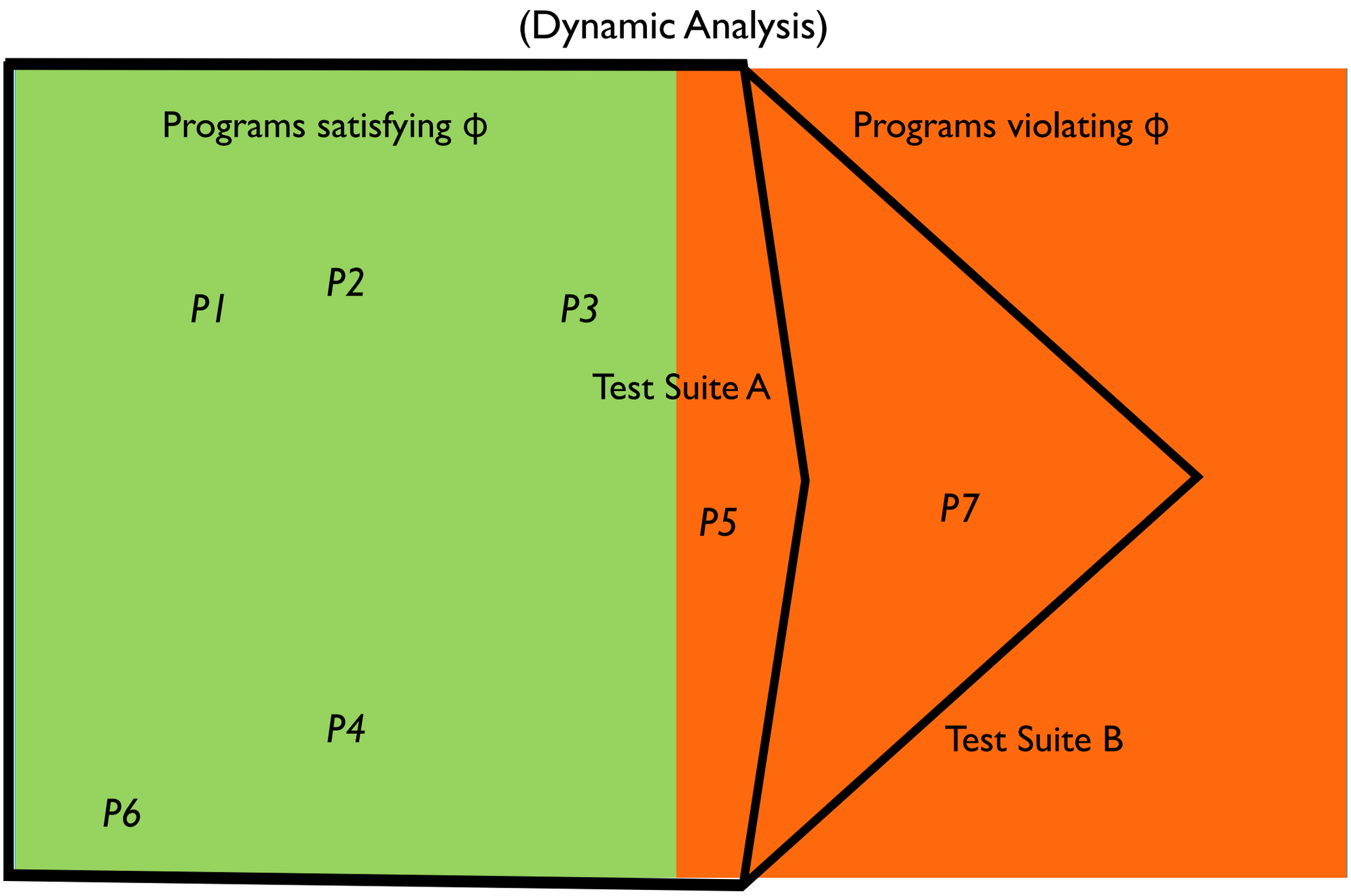
```
foo(5)
foo(3)
```

True positive!

Which Programs Satisfy ϕ?



Which Programs Satisfy ϕ?



Static vs Dynamic Analysis

- The main challenge of building a static analysis is choosing a **good abstraction function**

- The main challenge of performing a good dynamic analysis is selecting a **representative set of test cases.**

Which Programs Satisfy ϕ?

(Neural Analysis)

Contents

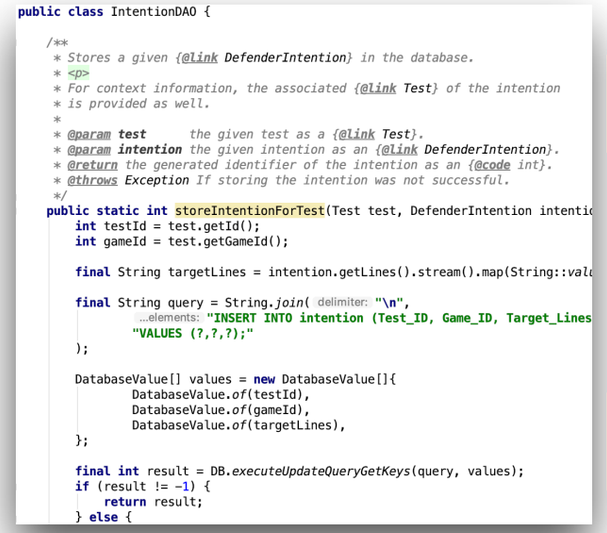
- What is Software Analysis?
- Course Organisation
- First Steps: Character Level Analysis

Software Analysis

Executable Program

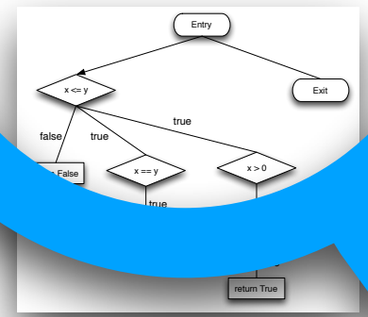
Execution Trace

```
0101011010
1010110101
0101101010
1010101010
1010111111
```



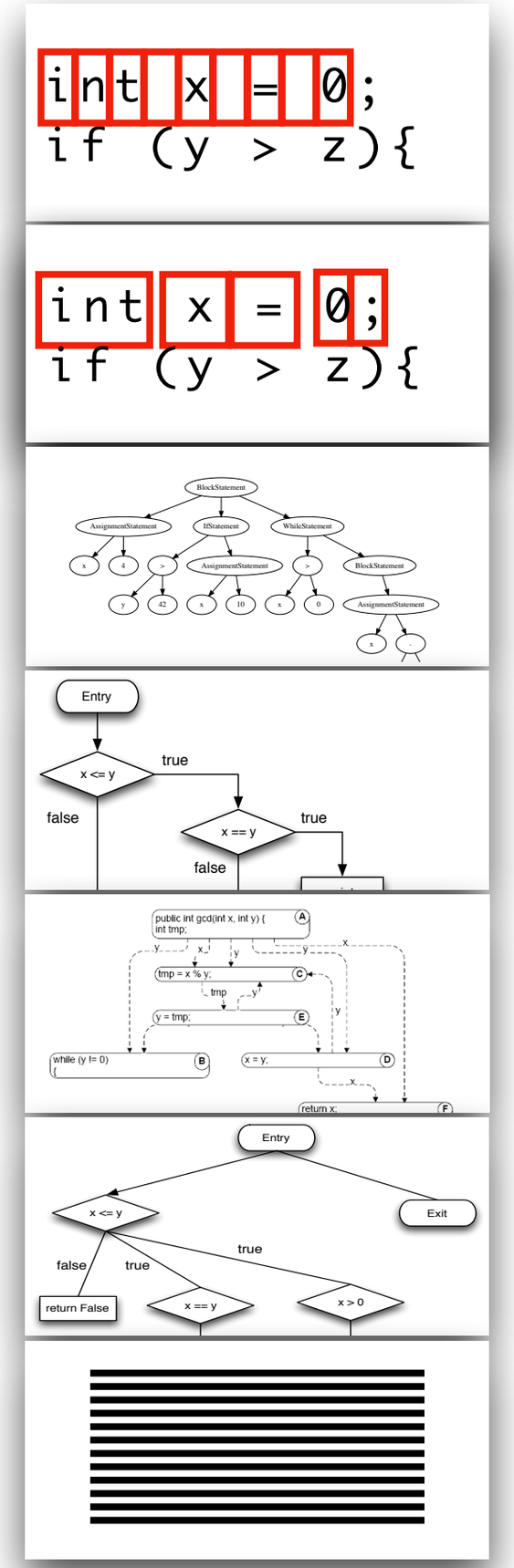
```
public class IntentionDAO {

    /**
     * Stores a given {@link DefenderIntention} in the database.
     * <p>
     * For context information, the associated {@link Test} of the intention
     * is provided as well.
     *
     * @param test      the given test as a {@link Test}.
     * @param intention the given intention as an {@link DefenderIntention}.
     * @return the generated identifier of the intention as an {@code int}.
     * @throws Exception If storing the intention was not successful.
     */
    public static int storeIntentionForTest(Test test, DefenderIntention intentio
        int testId = test.getId();
        int gameId = test.getGameId();

        final String targetLines = intention.getLines().stream().map(String::valu

        final String query = String.join( delimiter: "\n",
                ...elements: "INSERT INTO intention (Test_ID, Game_ID, Target_Lines
                "VALUES (?,?,?);"
        );

        DatabaseValue[] values = new DatabaseValue[]{
                DatabaseValue.of(testId),
                DatabaseValue.of(gameId),
                DatabaseValue.of(targetLines),
        };

        final int result = DB.executeUpdateQueryGetKeys(query, values);
        if (result != -1) {
            return result;
        } else {
```

Source Code



Internal Representation

Analysis Algorithm



Characters

Tokens

Syntax Trees

Control Flow

Data Flow

Dependency

Executions

1. Introduction, Character-level analysis
2. Token-level analysis: NLP for Source Code
3. Syntax Analysis
4. Controlflow Analysis
5. Dataflow Analysis
6. Dependence Graphs
7. Interprocedural Analysis
8. Symbolic Execution, Dynamic Symbolic Execution
9. Dynamic Analysis

Passing Requirements

- Registration on Campus Portal
- Assignments
  - Detailed grading criteria will be specified for each assignment
  - Tests for your implementation are part of the grading criteria!
- Exam
  - Oral or written: Depends on number of registrations
  - Date tbd
- _Both_ need to be passed, 50/50 weighting

Tentative Assignments

- Assignment 1 (Week 4):
  Code metrics + prediction

- Assignment 2 (Week 7):
  Interprocedural sign analysis

- Assignment 3 (Week 10):
  Dynamic program slicing

- Java (Non-negotiable)

Exercise Sessions

Patric Feldmeier

Wednesday 14:00-16:00

- Content: Assignment presentation, Q&A, and discussion

Information and Literature

- Lecture slides on StudIP

- Jupyter Notebooks with code examples

Contents

- What is Software Analysis?
- Course Organisation
- First Steps: Character Level Analysis

Code Clones

- *Code fragment*: A sequence of code lines
- *Code clone*: A code fragment in source files that is identical or similar to another
- Number 1 on Beck and Fowler's "Stink Parade of Bad Smells"
- Between 7%-23% of the code in a typical software system is cloned

Why Clone?

- Development Strategy
  - Simple reuse by Copy/Paste
  - Forking (reuse of similar solutions with the hope that they will be diverged significantly with the evolution of the system)
  - Delay in restructuring
- Maintenance Benefits
  - Risk in developing new code
  - Speed up maintenance
- Overcoming Limitations
  - Lack of reuse mechanism of programming languages
  - Significant efforts in writing reusable code
  - Time limit assigned to developers
  - Lack of ownership of the code to be reused
  - Developer's lack of knowledge in a problem domain

Drawbacks of Clones

- Increased probability of bug propagation
- Increased probability of introducing a new bug
- Increased probability of bad design
- Increased difficulty in system improvement/modification
- Increased maintenance cost
- Increased resource requirements

Advantages and Applications of Detecting Code Clones

- Improves software quality
- Detects library candidates
- Helps in program understanding
- Finds usage pattern
- Detects plagiarism and copyright infringement

Clone Types

- Type-1 clones
  Identical code fragments but may have some variations in whitespace, layout, and comments

- Type-2 clones
  Syntactically equivalent fragments with some variations in identifiers, literals, types, whitespace, layout and comments

- Type-3 clones
  Syntactically similar code with inserted, deleted, or updated statements

- Type-4 clones
  Semantically equivalent, but syntactically different

Clone Detection Strategies

- Text matching
  - No program structure is taken into consideration
  - Type-1 clones & some Type-2 clones
  - Exact string match vs Ambiguous match (e.g., Longest Common Subsequence match,)
  - —> See Jupyter Notebook
- Token sequence matching
  - No program structure is taken into account, either
  - Type-1 and Type-2 clones
- Graph matching
  - Type-1, Type-2, and Type-3 clones; some Type-4
  - Syntactic and semantic understanding (AST, CFG, PDG)
- Metrics based

Contents

- What is Software Analysis?
- Course Organisation
- Character Level Analysis